

SOLID Mapping — SALESTORM

1. Purpose

The SALESTORM architecture applies SOLID principles to keep the Inventory, Payment, Order, and external-integration components loosely coupled, testable, extensible, and maintainable.

The goal is not to use SOLID artificially. Each principle is applied where it solves a real design problem.

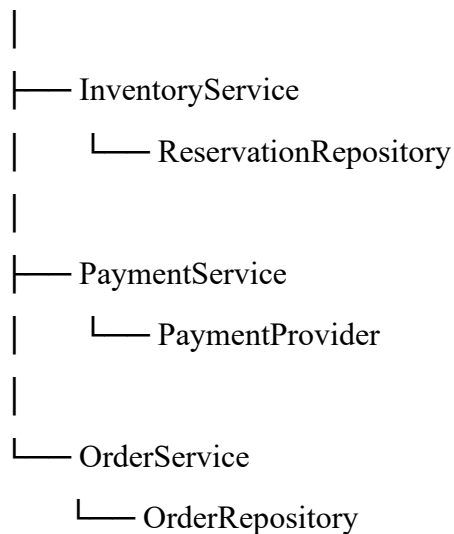
2. SRP — Single Responsibility Principle

A class/service should have one reason to change.

SALESTORM application

Instead of creating one giant CheckoutService, responsibilities are separated:

CheckoutService



Example

Component	Responsibility
-----------	----------------

InventoryService Inventory availability and reservation

PaymentService Payment lifecycle

OrderService Order lifecycle

NotificationService Customer notifications

ShipmentService Shipment lifecycle

Why?

If the payment provider changes, we should not need to modify inventory or order logic.

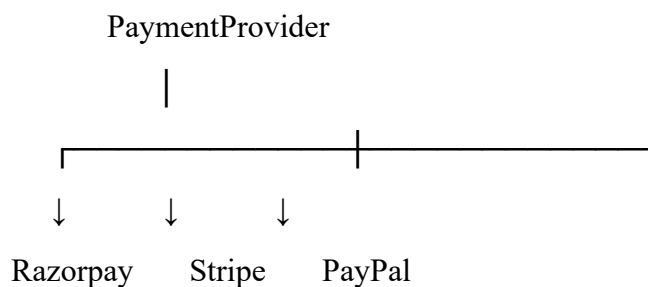
SALESTORM benefit: Reduced coupling and easier testing.

3. OCP — Open/Closed Principle

Open for extension, closed for modification.

SALESTORM should support adding new payment providers without changing the core payment workflow.

Example



The application depends on:

PaymentProvider

rather than directly depending on:

RazorpayPaymentProvider

Example interface

```
interface PaymentProvider {  
    initiatePayment()  
    checkStatus()  
    refund()  
}
```

New provider:

NewPaymentProvider implements PaymentProvider

without rewriting PaymentService.

SALESTORM benefit

New payment providers, pricing strategies, or delivery partners can be introduced with minimal modification.

4. LSP — Liskov Substitution Principle

Any implementation of an abstraction should be usable wherever that abstraction is expected.

For example:

PaymentProvider

↑

└─ RazorpayProvider

└─ StripeProvider

└─ MockPaymentProvider

PaymentService should work with any of them through the common interface.

PaymentService



PaymentProvider

The service should not need to know which provider is being used.

Example

During development:

MockPaymentProvider

During production:

RazorpayProvider

Both should satisfy the same contract.

SALESTORM benefit

Easy testing

Provider replacement

Reduced dependency on external vendors

5. ISP — Interface Segregation Principle

Classes should not be forced to depend on interfaces containing methods they don't need.

Instead of one huge interface:

```
PaymentProvider {  
    pay()  
    refund()  
    checkStatus()  
    generateInvoice()  
    sendNotification()  
    createShipment()  
}
```

Separate focused interfaces:

PaymentProcessor

PaymentStatusChecker

RefundProcessor

For example:

PaymentProcessor

→ initiatePayment()

PaymentStatusChecker

→ checkStatus()

RefundProcessor

→ refund()

SALESTORM benefit

Smaller interfaces mean:

less coupling

easier mocking

simpler testing

clearer responsibilities

6. DIP — Dependency Inversion Principle

High-level modules should depend on abstractions, not concrete implementations.

Instead of:

PaymentService



RazorpayAPI

use:

PaymentService



PaymentProvider



RazorpayProvider

Similarly:

InventoryService



InventoryRepository



MySQLInventoryRepository

and:

OrderService



OrderRepository



MySQLOrderRepository

SALESTORM benefit

The business logic doesn't become tightly coupled to:

MySQL implementation

Payment provider

Message broker

External delivery partner

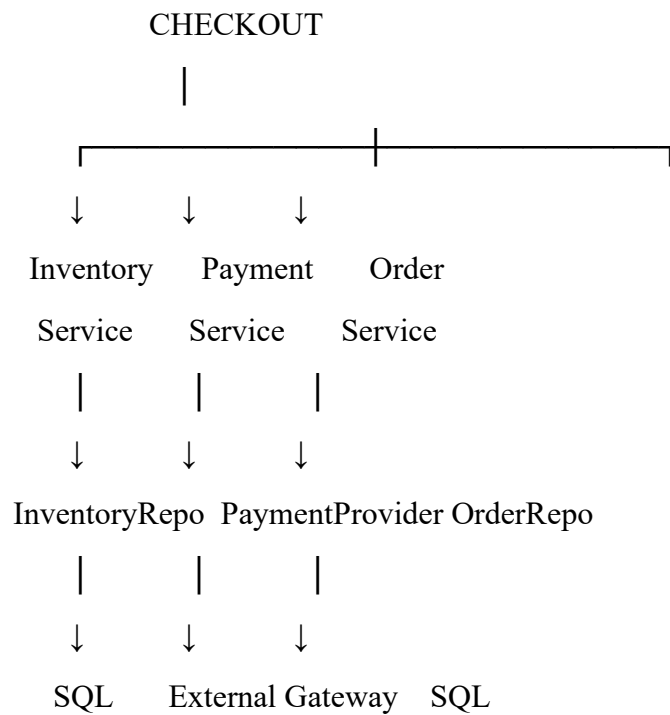
This makes the system easier to test and evolve.

7. Complete SOLID Mapping

Principle	SALESTORM Application	Benefit
SRP	Separate Inventory, Payment, Order and Notification responsibilities	Lower coupling
OCP	Payment/Delivery strategies use interfaces	Easy extension
LSP	Payment providers implement common provider contract	Provider interchangeability
ISP	Separate payment/status/refund interfaces	Focused dependencies
DIP	Services depend on repositories/provider abstractions	Testability + flexibility

8. SOLID in the Critical Purchase Flow

This is a particularly strong way to explain it to the jury:



Each service owns a clear responsibility, while abstractions prevent the business logic from being tied to infrastructure implementations.